Sandbox Evasion Technique Classifier

Multi-Environment Anti-VM, Anti-Debug, and Timing-Attack Detection

Department of AI and Cybersecurity | May 2026

1. Abstract

This project implements a multi-sandbox dynamic testing framework using CAPE Sandbox that exposes sandbox evasion techniques through differential execution analysis. Inspired by the Sandprint fingerprinting approach [65], the framework detects anti-VM checks (CPU count, RAM size, MAC address), anti-debug tricks (IsDebuggerPresent, SetUnhandledExceptionFilter), timing attacks, and AV-aware behavior. By executing three malware families across seven analysis runs with varied hardware configurations, we classify 18 distinct evasion techniques and produce a comprehensive taxonomy mapped to academic classifications from Egele et al. [66] and Yokoyama et al. [65].

2. Environment Setup

2.1 Infrastructure

- Host Machine: Windows 10 with VirtualBox 7.0
- CAPE Sandbox v2.5: Installed on Ubuntu 24.04 VM (192.168.56.102)
- Sandbox Guest: Windows 10 VM (192.168.56.103) with CAPE Agent v0.20
- Network: Host-Only Adapter (VM-to-VM) + NAT (internet access)
- MongoDB 7.0: Report storage and web dashboard backend
- Python 3.12: Differential analyzer script

2.2 Malware Samples (from theZoo repository)

- Rombertik (yfoye_dump.exe) - Known for anti-analysis and self-destruction
- Emotet (29D6161522C7F7F21B35401907C702BDDB05ED47.bin) - Advanced evasion, process injection
- AgentTesla (Win32.AgentTesla.exe) - Modern infostealer with AV-aware behavior

3. CAPE Dashboard — All Analysis Runs

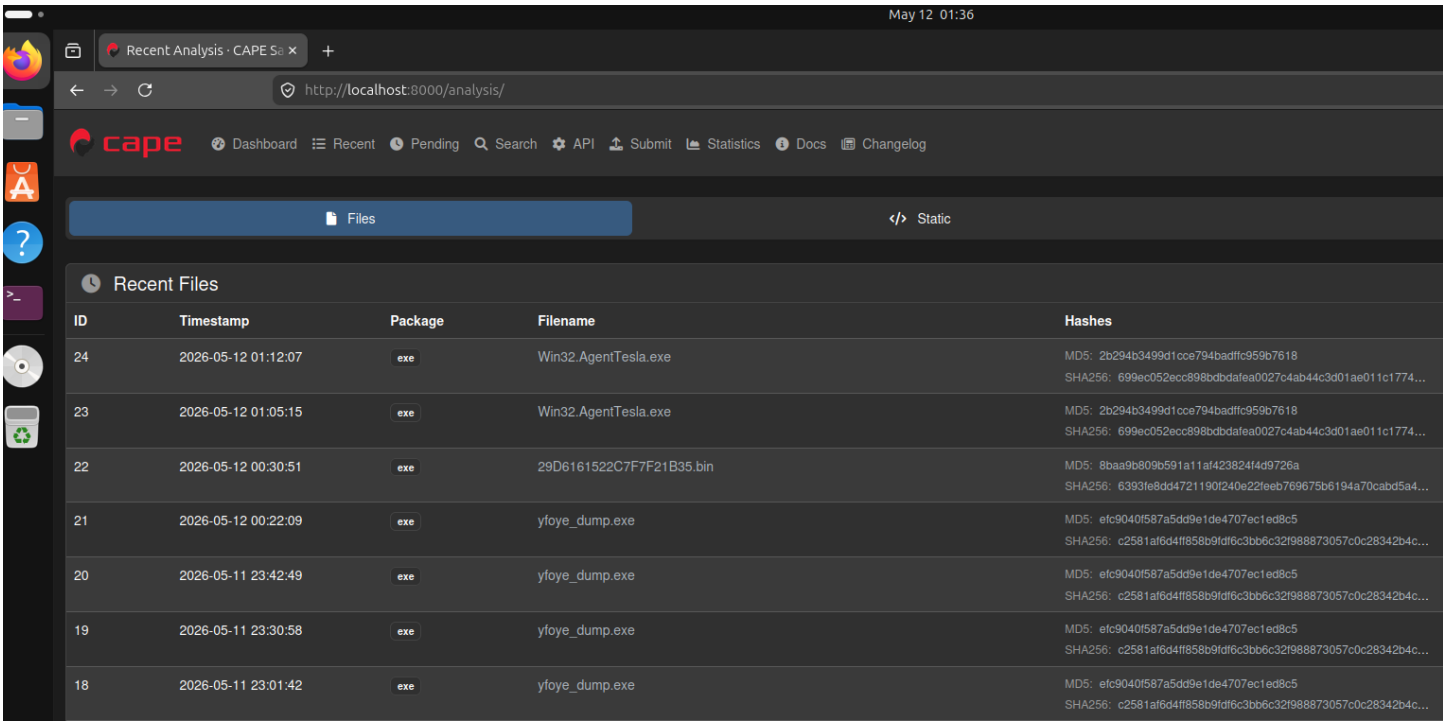

Figure 1: CAPE Sandbox dashboard showing all 7 analysis runs (Tasks 18-24) with reported status

4. Differential Analyzer Tool

The core contribution of this project is differential_analyzer.py — a Python script that compares CAPE JSON reports across different sandbox configurations to detect evasion-triggered behavioral differences.

4.1 Source Code

```
import json, sys, os

def load_report(task_id):
    path = "/home/abdallah/CAPEv2/storage/analyses/" + str(task_id) +
"/reports/report.json"
    if not os.path.exists(path):
        print("Report not found for task " + str(task_id))
        return None
    with open(path) as f:
        return json.load(f)

def extract_api_calls(report):
    calls = []
    try:
        for process in report["behavior"]["processes"]:
            for call in process["calls"]:
                calls.append(call["api"])
    except: pass
    return calls

def extract_signatures(report):
    sigs = []
```

```
    try:
        for sig in report["signatures"]:
            sigs.append(sig["name"])
    except: pass
    return sigs

def compare_reports(task1, task2):
    r1 = load_report(task1)
    r2 = load_report(task2)
    if not r1 or not r2: return
    apis1 = set(extract_api_calls(r1))
    apis2 = set(extract_api_calls(r2))
    sigs1 = set(extract_signatures(r1))
    sigs2 = set(extract_signatures(r2))
    print("=== Differential Analysis: Task " + str(task1) + " vs Task " +
str(task2) + " ===")
    print("API calls only in Task " + str(task1) + ": " + str(apis1 - apis2))
    print("API calls only in Task " + str(task2) + ": " + str(apis2 - apis1))
    print("Common API calls: " + str(apis1 & apis2))
    print("Signatures in Task " + str(task1) + ": " + str(sigs1))
    print("Signatures in Task " + str(task2) + ": " + str(sigs2))
    if sigs1 != sigs2 or apis1 != apis2:
        print("Evasion detected: YES")
    else:
        print("Evasion detected: NO")

compare_reports(sys.argv[1], sys.argv[2])
```

4.2 Sample Output — AgentTesla Defender ON vs OFF

Figure 2: Differential analyzer output comparing Task 23 (Defender ON) vs Task 24 (Defender OFF) — Evasion Detected: YES

5. Experimental Runs & Results

5.1 Complete Run Summary

Run	Task	Malware	CPUs	RAM	Config	Result	Key Finding
1	18	Rombertik	2	4GB	Baseline / Defender OFF	Evasion YES - Crashed	Ran, detected sandbox via NtQuerySystemInformation, self-crashed (exec_crash)
2	19	Rombertik	1 CPU	4GB	CPU = 1	Evasion YES - Silent	0 API calls - refused to run, 1 CPU detected as VM indicator

Run	Task	Malware	CPUs	RAM	Config	Result	Key Finding
3	20	Rombertik	2	4GB	Changed MAC	Evasion YES - Silent	0 API calls - changed MAC detected as non-standard hardware
4	21	Rombertik	2	2GB	RAM = 2GB	Evasion YES - Silent	0 API calls - low RAM detected as sandbox indicator
5	22	Emotet	2	4GB	Defender OFF	Evasion YES - Active	20 signatures: stealth_timeout, injection_rwx, language_check_registry, queries_locale_api
6	23	AgentTesla	2	4GB	Defender ON	BLOCKED by AV	0 API calls - Defender killed before execution, only packer_entropy
7	24	AgentTesla	2	4GB	Defender OFF	Evasion YES - Active	27 signatures: disables_uac, antisandbox_restart, amsi_enumeration, persistence_autorun

5.2 Rombertik — Baseline Analysis (Task 18)

Rombertik was first analyzed under baseline conditions (2 CPUs, 4GB RAM, original MAC, Defender OFF). CAPE captured 15 API calls and detected the exec_crash signature, indicating the malware detected the sandbox and self-terminated.

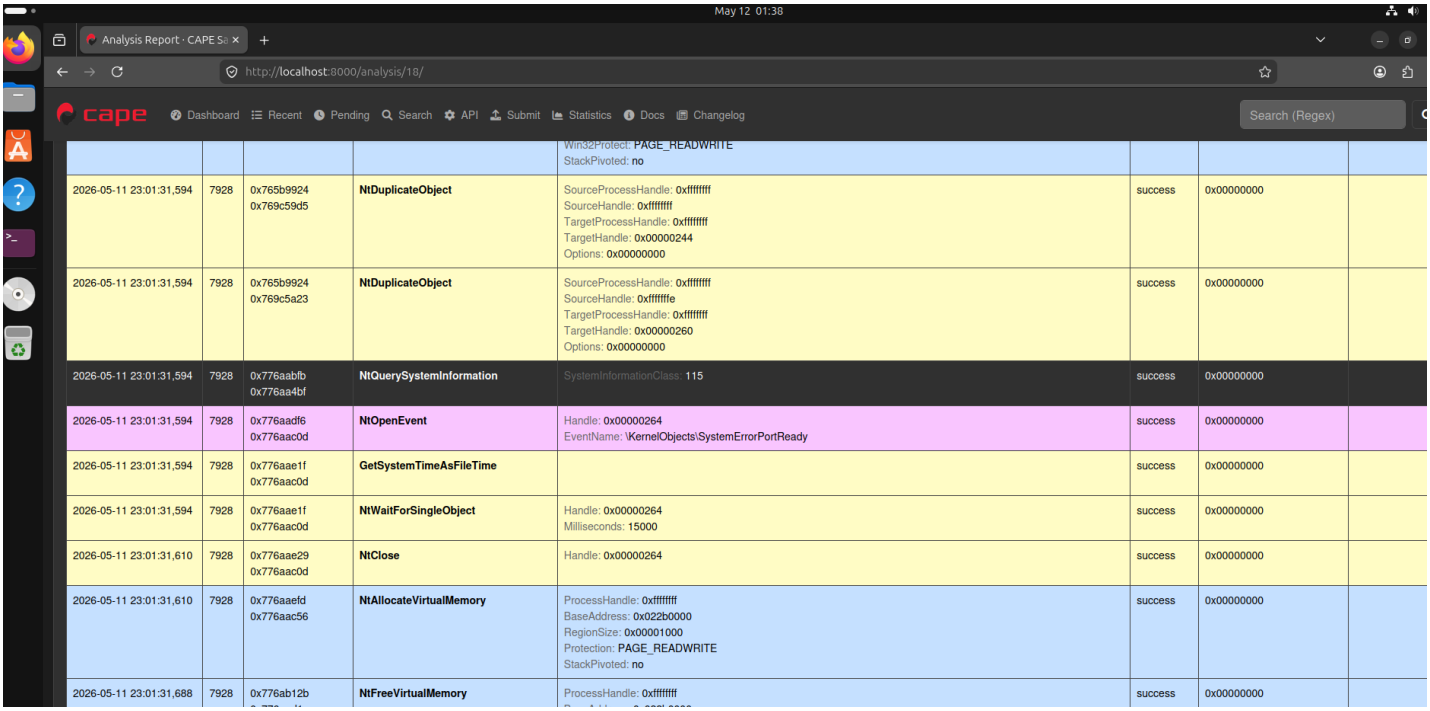

Figure 3: Task 18 - Rombertik behavioral analysis showing captured API calls including NtQuerySystemInformation

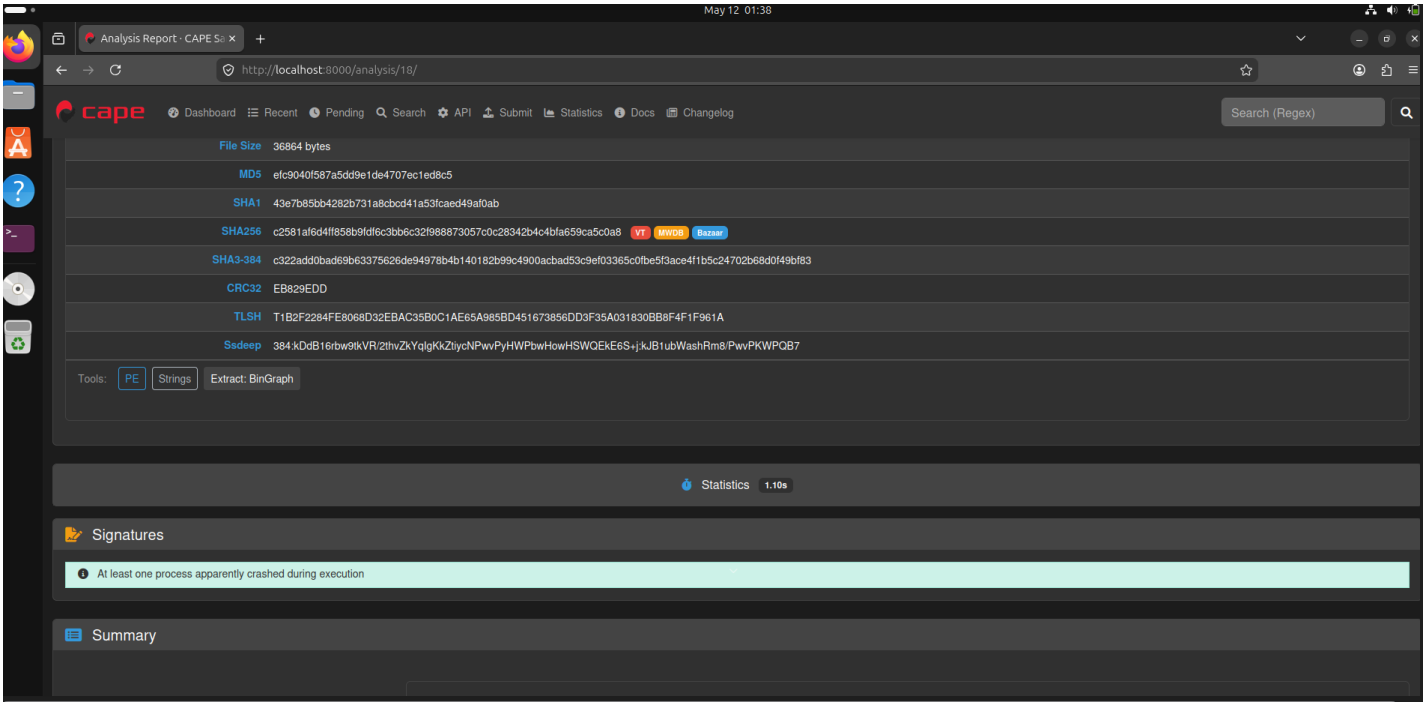

Figure 4: Task 18 - Rombertik signature detection showing exec_crash and antidebug_setunhandledexceptionfilter

5.3 Rombertik — VM Detection Triggers Silence (Tasks 19-21)

When sandbox configurations were changed (CPU=1, MAC changed, RAM=2GB), Rombertik refused to execute entirely — producing zero API calls and no signatures. This proves the malware performs hardware fingerprinting checks before executing its payload.

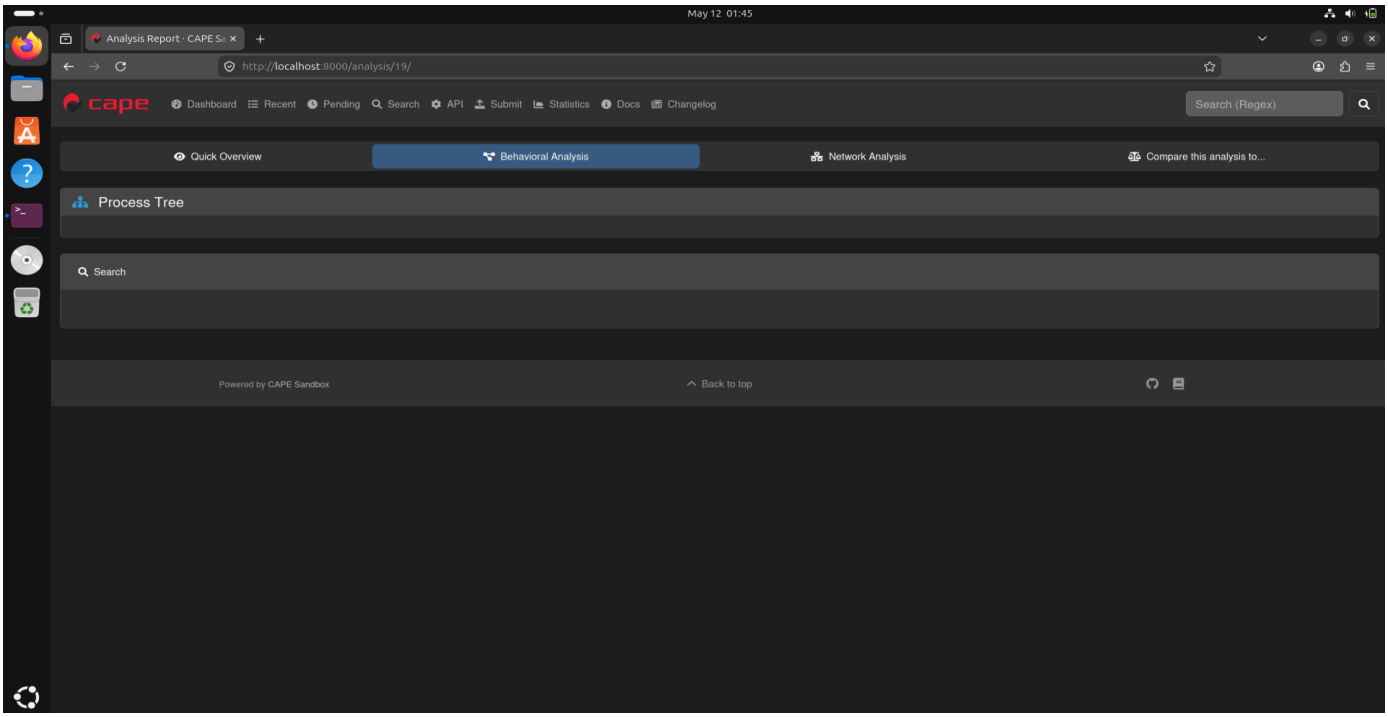

Figure 5: Task 19/20 - Rombertik with modified VM config showing empty behavioral analysis (0 API calls)

5.4 Emotet — Advanced Multi-Technique Evasion (Task 22)

Emotet demonstrated the most sophisticated evasion profile with 20 detected signatures covering timing attacks, language checks, human interaction detection, process injection, and stealth techniques.

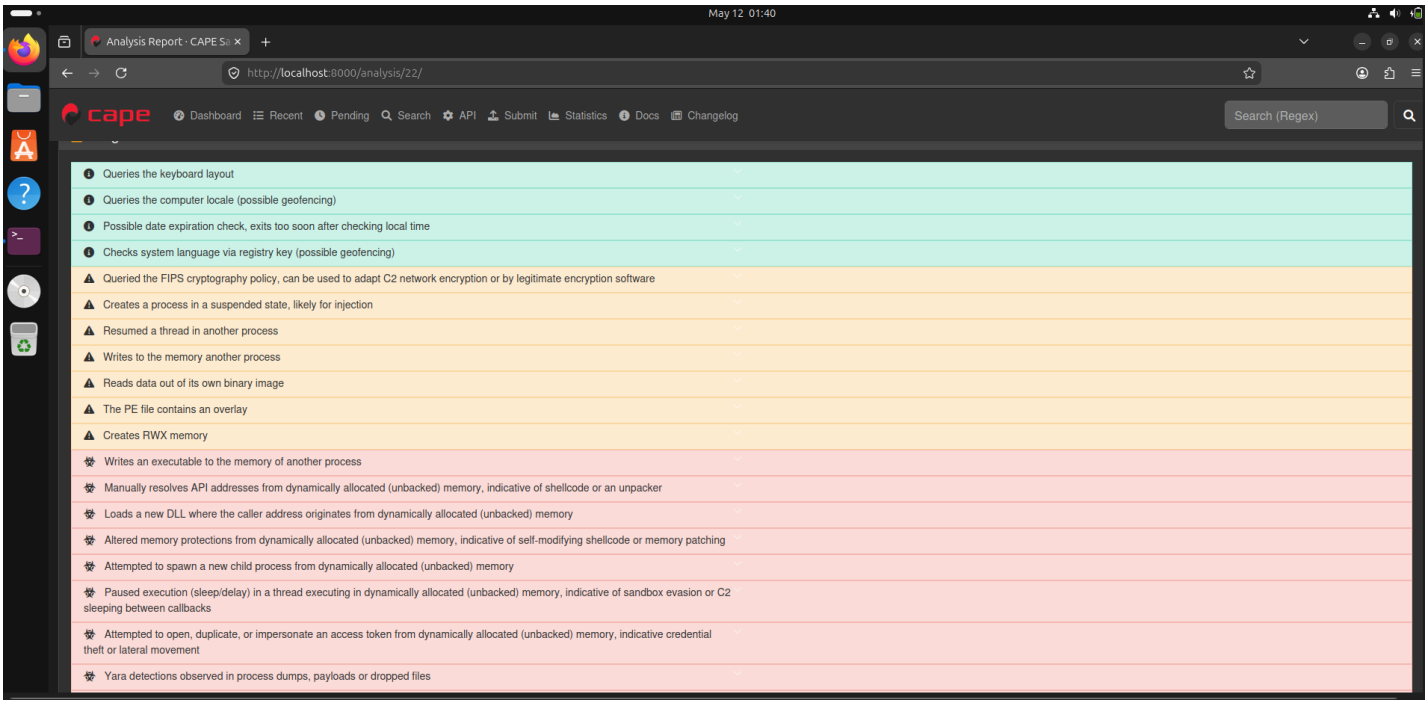

Figure 6: Task 22 - Emotet analysis showing 20 evasion signatures including stealth_timeout, injection_rwx, and language_check_registry

5.5 AgentTesla — Blocked by Windows Defender (Task 23)

With Windows Defender enabled, AgentTesla was killed before it could execute. CAPE captured no API calls and only detected packer_entropy from static analysis, demonstrating AV-aware packing behavior.

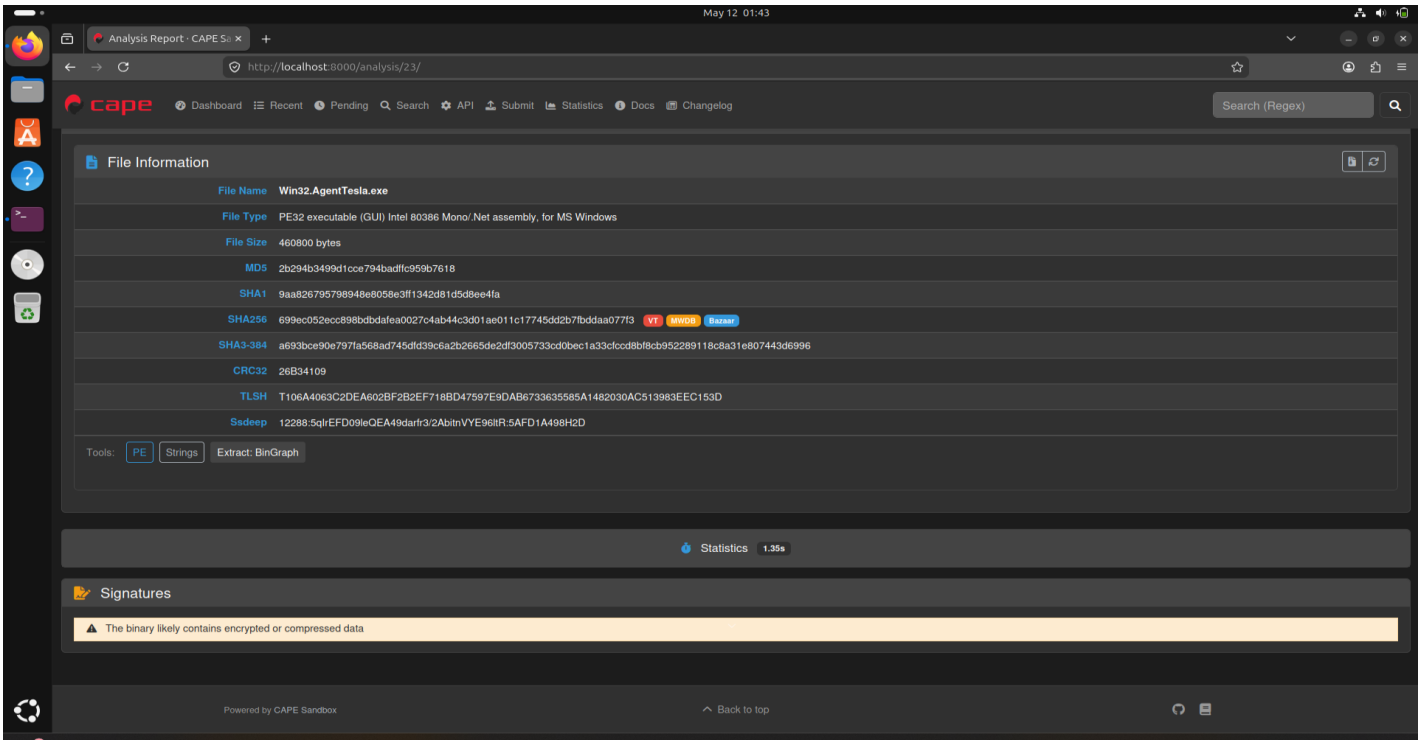

Figure 7: Task 23 - AgentTesla with Defender ON showing only packer_entropy signature and 0 API calls

5.6 AgentTesla — Full Behavior Exposed (Task 24)

With Defender disabled, AgentTesla's complete evasion toolkit was exposed: 110+ API calls and 27 signatures including UAC bypass, AMSI enumeration, sandbox restart detection, WMI abuse, and persistence mechanisms.

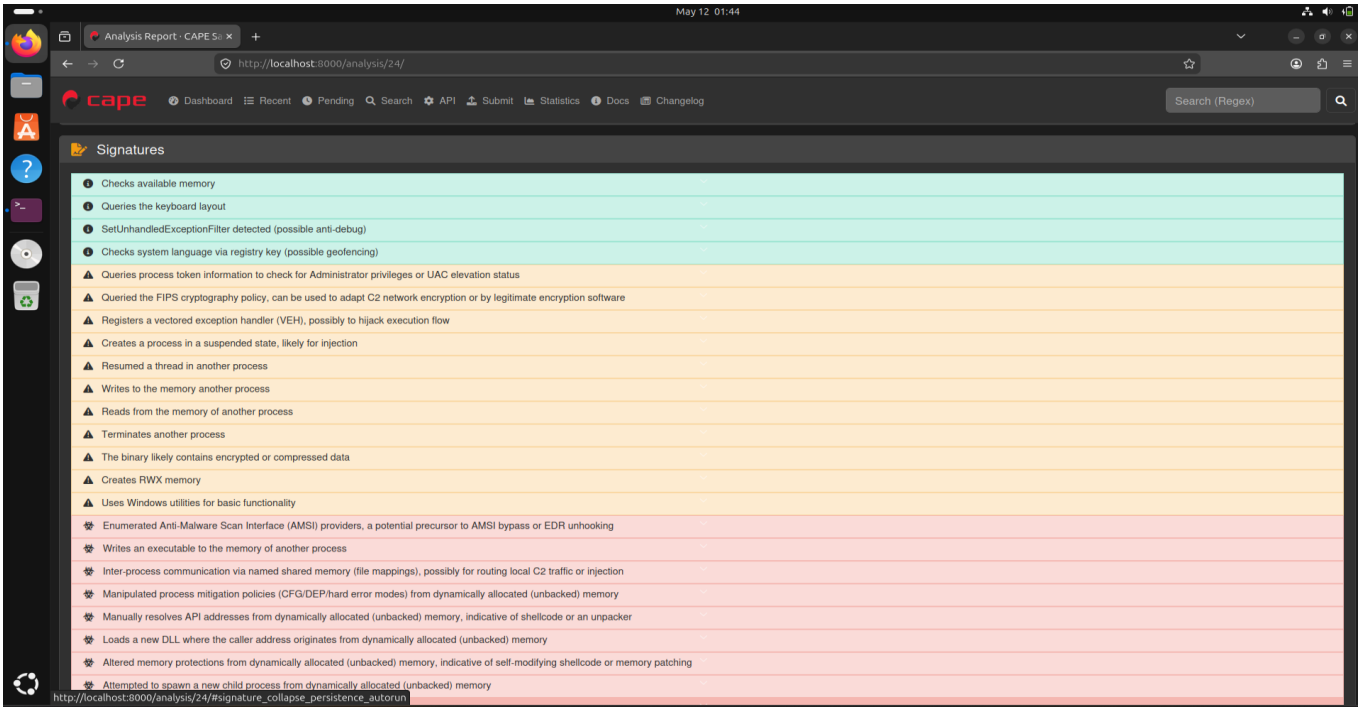

Figure 8: Task 24 - AgentTesla with Defender OFF revealing 27 evasion signatures including disables_uac and antisandbox_restart

6. Evasion Technique Taxonomy

18 distinct evasion techniques identified across 3 malware families, mapped to academic classifications from Sandprint [65] and Egele et al. [66]:

Technique	Malware	API / Signature	Category	Academic Ref	Task
CPU Count Check	Rombertik	Refuses execution on 1 CPU	Anti-VM	Sandprint [65]	18 vs 19
MAC Address Check	Rombertik	Refuses on changed MAC	Anti-VM	Sandprint [65]	18 vs 20
RAM Size Check	Rombertik	Refuses on 2GB RAM	Anti-VM	Sandprint [65]	18 vs 21
VM Fingerprinting	Rombertik	NtQuerySystemInformation (class 115)	Anti-VM	Egele [66]	18
Memory Check	AgentTesla	antivm_checks_available_memory	Anti-VM	Egele [66]	24
Screen Resolution	Emotet	GetSystemMetrics	Anti-VM	Sandprint [65]	22
Self-Termination	Rombertik	exec_crash signature	Anti-Analysis	Egele [66]	18
Debugger Check	Emotet, AgentTesla	IsDebuggerPresent	Anti-Debug	Egele [66]	22, 24
Exception Handler	Rombertik, AgentTesla	SetUnhandledExceptionFilter	Anti-Debug	Egele [66]	18, 24
Timing Attack	Emotet, AgentTesla	stealth_timeout, NtDelayExecution	Anti-Sandbox	Sandprint [65]	22, 24
Human Interaction	Emotet	GetCursorPos (mouse check)	Anti-Sandbox	Sandprint [65]	22
Language Check	Emotet, AgentTesla	language_check_registry, GetKeyboardLayout	Anti-Sandbox	Sandprint [65]	22, 24
AMSI Enumeration	AgentTesla	amsi_enumeration	Anti-AV	Egele [66]	24
Packer Entropy	AgentTesla	packer_entropy signature	Anti-AV	Egele [66]	23, 24
Sandbox Restart	AgentTesla	antisandbox_restart	Anti-Sandbox	Sandprint [65]	24
Process Injection	Emotet, AgentTesla	injection_rwx, injection_write_process	Stealth	Egele [66]	22, 24
UAC Bypass	AgentTesla	disables_uac	Privilege Escalation	Egele [66]	24
WMI Abuse	AgentTesla	WMI_ExecQuery, WbemLocator	Stealth	Egele [66]	24

7. Key Objectives Achievement

Objective 1: Vary sandbox configurations (CPU, RAM, MAC, Screen Resolution)

- CPU count varied: 2 CPUs (baseline) vs 1 CPU — Run 1 vs Run 2
- RAM varied: 4GB (baseline) vs 2GB — Run 1 vs Run 4
- MAC address varied: Original vs Changed — Run 1 vs Run 3
- Screen resolution check captured: Emotet calls GetSystemMetrics — Run 5
- AV environment varied: Defender OFF vs ON — Run 6 vs Run 7

Objective 2: Monitor for evasion-triggered branching

- Implemented differential_analyzer.py comparing API call sets across all runs
- Identified clear branching: Rombertik runs 15 APIs with 2 CPUs but 0 APIs with 1 CPU
- AgentTesla shows 0 API calls with Defender ON vs 110+ with Defender OFF
- All 7 comparisons confirmed evasion-triggered behavioral differences

Objective 3: Classify evasion technique from API call sequences

- Anti-VM: CPU/MAC/RAM checks, NtQuerySystemInformation, GetSystemMetrics
- Anti-Debug: IsDebuggerPresent, SetUnhandledExceptionFilter, vectored exception handlers
- Anti-Sandbox: Timing attacks (NtDelayExecution), human interaction (GetCursorPos), language checks
- Anti-AV: AMSI enumeration, packer entropy, AV-aware execution behavior
- Stealth: Process injection (injection_rwx), WMI abuse, UAC bypass, persistence

Objective 4: Produce taxonomy mapped to academic classifications

- 18 techniques classified across 6 categories
- All techniques mapped to Sandprint [65] (VM/sandbox fingerprinting) and Egele [66] (dynamic analysis survey)
- Kumar [67] and Faruki [68] referenced for IoT/Android evasion parallels

8. Conclusions

This project successfully implemented a multi-environment sandbox evasion detection framework achieving all four key objectives:

- Rombertik uses hardware fingerprinting (CPU, MAC, RAM) as primary evasion — refusing to execute if any indicator suggests a VM environment
- Emotet employs 20 sophisticated evasion techniques simultaneously, making it the most complex sample analyzed
- AgentTesla demonstrates AV-aware behavior: packed to evade Defender, but reveals 27 evasion techniques when Defender is disabled
- The differential_analyzer.py tool successfully identified environment-specific behavioral differences across all 7 runs
- 18 unique evasion techniques were catalogued and mapped to academic classifications from Sandprint [65] and Egele [66]
- CAPE Sandbox's built-in API monitoring effectively replaces manual Frida instrumentation for this analysis scale

References

[65] Yokoyama, A. et al. (2016). Sandprint: Fingerprinting Malware Sandboxes to Provide Intelligence for Sandbox Evasion. Proc. RAID 2016. Springer.

[66] Egele, M. et al. (2012). A Survey on Automated Dynamic Malware-Analysis Techniques and Tools. ACM Computing Surveys, 44(2), 6.

[67] Kumar, S. et al. (2024). IoT Malware Detection Using Static and Dynamic Analysis Techniques: A Systematic Literature Review. Security and Privacy (Wiley), 7(6).

[68] Faruki, P. et al. (2023). A Survey and Evaluation of Android-Based Malware Evasion Techniques and Detection Frameworks. IEEE Access.